

AI ASSISTED CODING

ASSIGNMENT-11.2

NAME: Geethanjali

HTNO: 2303A51005

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
1 #generate a code to demonstrate Stack and its operation
2 class Stack:
3     def __init__(self):
4         self.stack = []
5
6     def push(self, item):
7         self.stack.append(item)
8         print(f'Pushed {item} to stack.')
9
10    def pop(self):
11        if not self.is_empty():
12            item = self.stack.pop()
13            print(f'Popped {item} from stack.')
14            return item
15        else:
16            print('Stack is empty. Cannot pop.')
17            return None
18
19    def peek(self):
20        if not self.is_empty():
21            item = self.stack[-1]
22            print(f'Top item is {item}.')
23            return item
24        else:
25            print('Stack is empty. Cannot peek.')
26            return None
27
28    def is_empty(self):
29        return len(self.stack) == 0
30
31    def size(self):
32        size = len(self.stack)
33        print(f'Stack size is {size}.')
34        return size
35
36    # Demonstrating Stack operations
37    if __name__ == "__main__":
38        stack = Stack()
39        stack.push(1)
40        stack.push(2)
41        stack.push(3)
42        stack.peek()
43        stack.size()
44        stack.pop()
45        stack.peek()
46        stack.size()
47        stack.pop()
48        stack.pop() # Attempt to pop from an empty stack
```

```
demonstrate.py > ...
1 #generate a code to demonsttrare stack and its operation
2 class Stack:
3     def __init__(self):
4         self.stack = []
5
6     def push(self, item):
7         self.stack.append(item)
8         print(f'Pushed {item} to stack.')
9
10    def pop(self):
11        if not self.is_empty():
12            item = self.stack.pop()
13            print(f'Popped {item} from stack.')
14            return item
15        else:
16            print('Stack is empty. Cannot pop.')
17            return None
18
19    def peek(self):
20        if not self.is_empty():
21            item = self.stack[-1]
22            print(f'Top item is {item}.')
23            return item
24        else:
25            print('Stack is empty. Cannot peek.')
26            return None
27
28    def is_empty(self):
29
geethanjali@Geethanjali AI % /usr/local/bin/python3 ~/Users/geethanjali/Desktop/AI/ demonstrate .py"
Pushed 2 to stack.
Top item is 3.
Stack size is 3.
Popped 3 from stack.
Top item is 2.
Stack size is 2.
Popped 2 from stack.
Popped 1 from stack.
Stack is empty. Cannot pop.
geethanjali@Geethanjali AI %
```

Task Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

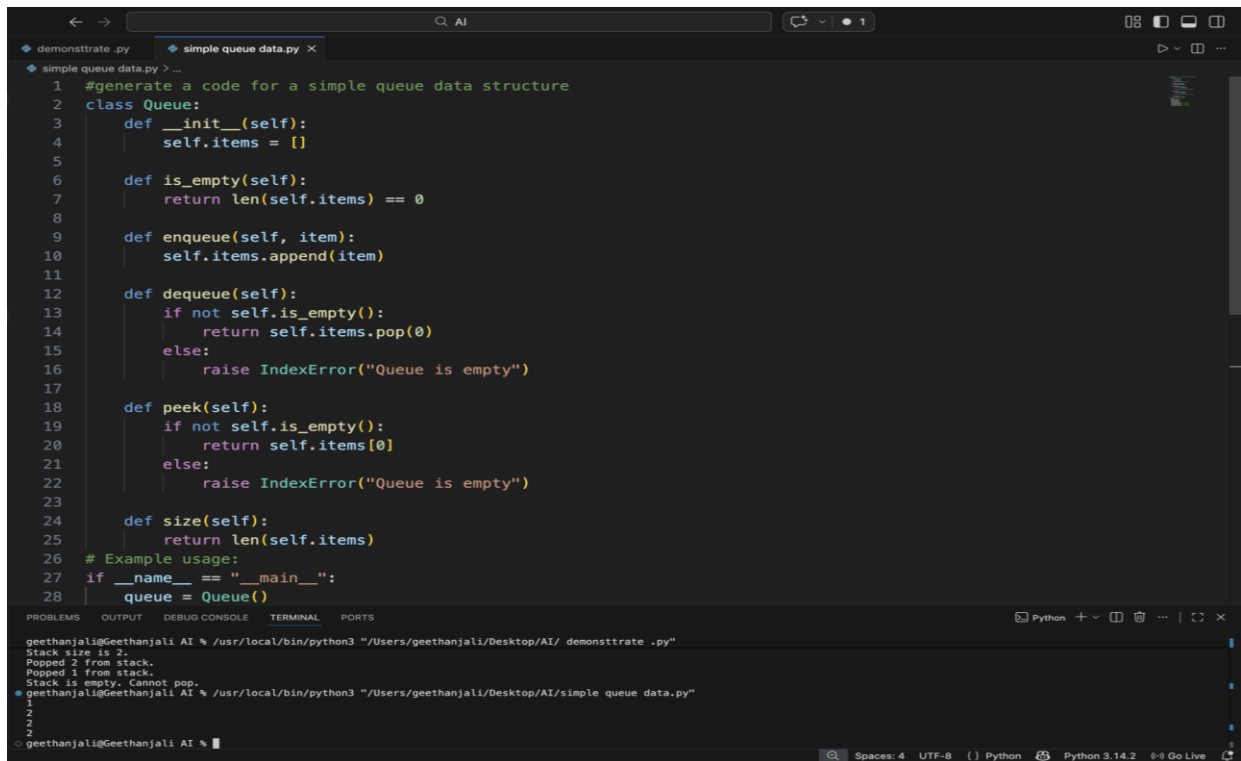
Sample Input Code: class

Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.



```
1 #generate a code for a simple queue data structure
2 class Queue:
3     def __init__(self):
4         self.items = []
5
6     def is_empty(self):
7         return len(self.items) == 0
8
9     def enqueue(self, item):
10        self.items.append(item)
11
12    def dequeue(self):
13        if not self.is_empty():
14            return self.items.pop(0)
15        else:
16            raise IndexError("Queue is empty")
17
18    def peek(self):
19        if not self.is_empty():
20            return self.items[0]
21        else:
22            raise IndexError("Queue is empty")
23
24    def size(self):
25        return len(self.items)
26
27 # Example usage:
28 if __name__ == "__main__":
29     queue = Queue()
30     queue.enqueue(1)
31     queue.enqueue(2)
32     print(queue.dequeue())
33     print(queue.dequeue())
34     print(queue.is_empty())
35     print(queue.size())
```

geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/ demonstrate .py"

Stack size is 2.

Popped 2 from stack.

Popped 1 from stack.

Stack is empty. Cannot pop.

geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/simple queue data.py"

1

2

2

geethanjali@Geethanjali AI %

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code: class

Node:

pass class

LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

```
1 # generate a code for a linked list and demonstrate its operations
2 class Node:
3     def __init__(self, data):
4         self.data = data
5         self.next = None
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9     def append(self, data):
10        new_node = Node(data)
11        if not self.head:
12            self.head = new_node
13            return
14        last_node = self.head
15        while last_node.next:
16            last_node = last_node.next
17        last_node.next = new_node
18    def print_list(self):
19        current_node = self.head
20        while current_node:
21            print(current_node.data)
22            current_node = current_node.next
23    def delete_node(self, key):
24        current_node = self.head
25        if current_node and current_node.data == key:
26            self.head = current_node.next
27            current_node = None
28            return
29        prev_node = None
30        while current_node and current_node.data != key:
31            prev_node = current_node
32            current_node = current_node.next
33        if not current_node:
34            return
35        prev_node.next = current_node.next
36        current_node = None
37 # Demonstrating the operations of the linked list
38 if __name__ == "__main__":
39     linked_list = LinkedList()
40     linked_list.append(1)
41     linked_list.append(2)
42     linked_list.append(3)
43     print("Linked List after appending 1, 2, 3:")
44     linked_list.print_list()
45     linked_list.delete_node(2)
46     print("Linked List after deleting node with data 2:")
47     linked_list.print_list()
```

geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/# linked list and .py"

3

geethanjali@Geethanjali AI % /usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/# linked list and .py"

Linked List after appending 1, 2, 3:

1

2

3

Linked List after deleting node with data 2:

1

3

geethanjali@Geethanjali AI %

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

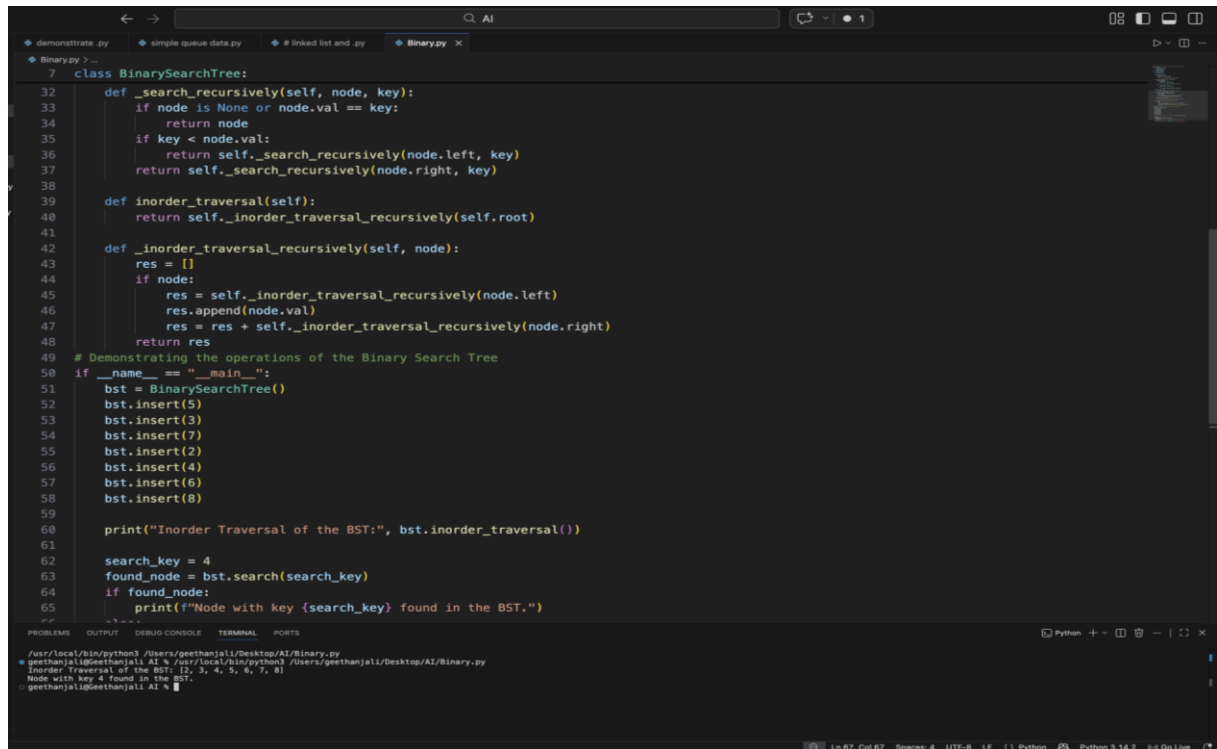
Sample Input Code:

```
class BST:
```

```
pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.



```
class BinarySearchTree:
    def _search_recursively(self, node, key):
        if node is None or node.val == key:
            return node
        if key < node.val:
            return self._search_recursively(node.left, key)
        return self._search_recursively(node.right, key)

    def inorder_traversal(self):
        return self._inorder_traversal_recursively(self.root)

    def _inorder_traversal_recursively(self, node):
        res = []
        if node:
            res = self._inorder_traversal_recursively(node.left)
            res.append(node.val)
            res = res + self._inorder_traversal_recursively(node.right)
        return res

# Demonstrating the operations of the Binary Search Tree
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)

    print("Inorder Traversal of the BST:", bst.inorder_traversal())

    search_key = 4
    found_node = bst.search(search_key)
    if found_node:
        print(f"Node with key {search_key} found in the BST.")
```

Output:

```
/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/Binary.py
Inorder Traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
Node with key 4 found in the BST.
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods. Sample Input Code: class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```
1 #generate a code of data structure for Binary Search Tree (BST)
2 class TreeNode:
3     def __init__(self, key):
4         self.left = None
5         self.right = None
6         self.val = key
7 class BinarySearchTree:
8     def __init__(self):
9         self.root = None
10
11     def insert(self, key):
12         if self.root is None:
13             self.root = TreeNode(key)
14         else:
15             self._insert_recursively(self.root, key)
16
17     def _insert_recursively(self, node, key):
18         if key < node.val:
19             if node.left is None:
20                 node.left = TreeNode(key)
21             else:
22                 self._insert_recursively(node.left, key)
23         else:
24             if node.right is None:
25                 node.right = TreeNode(key)
26             else:
27                 self._insert_recursively(node.right, key)
28
29     def search(self, key):
30         return self._search_recursively(self.root, key)
31
32     def _search_recursively(self, node, key):
33         if node is None or node.val == key:
34             return node
35         if key < node.val:
36             return self._search_recursively(node.left, key)
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

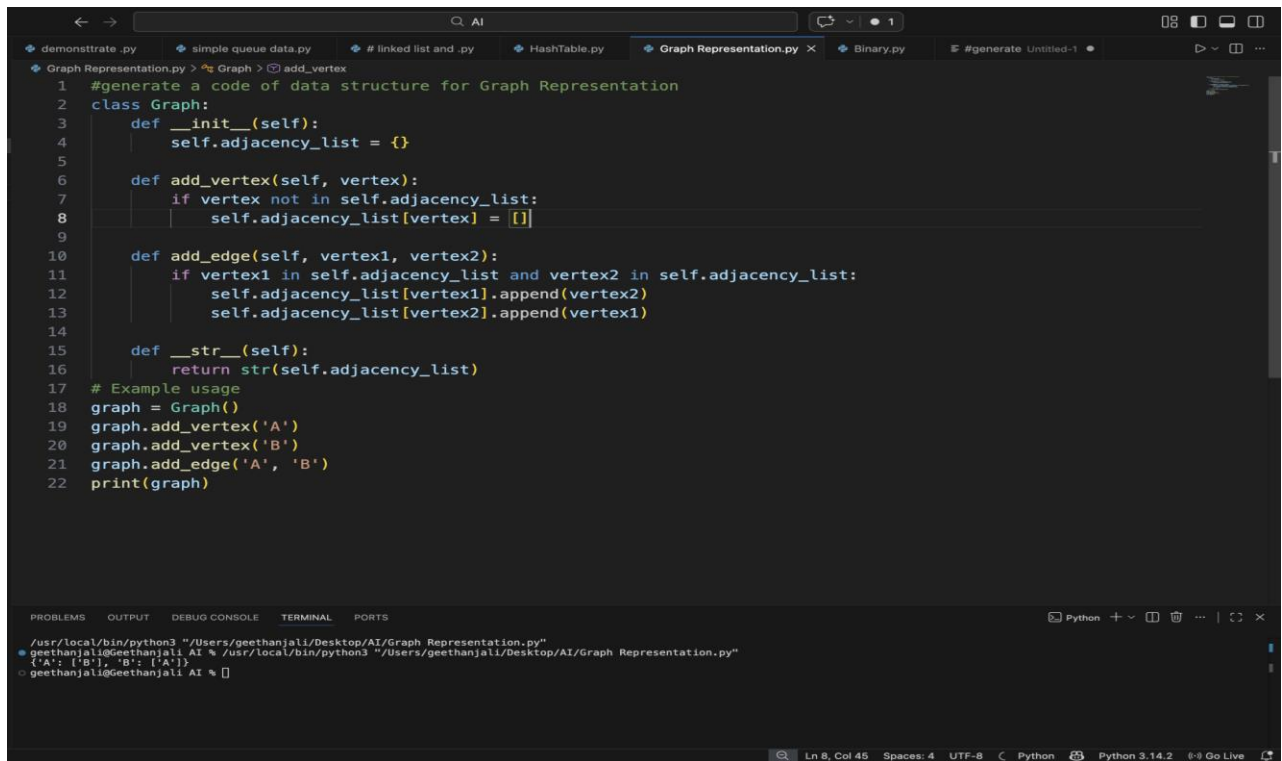
Sample Input Code: class

Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.



```
1 #generate a code of data structure for Graph Representation
2 class Graph:
3     def __init__(self):
4         self.adjacency_list = {}
5
6     def add_vertex(self, vertex):
7         if vertex not in self.adjacency_list:
8             self.adjacency_list[vertex] = []
9
10    def add_edge(self, vertex1, vertex2):
11        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
12            self.adjacency_list[vertex1].append(vertex2)
13            self.adjacency_list[vertex2].append(vertex1)
14
15    def __str__(self):
16        return str(self.adjacency_list)
17
18 # Example usage
19 graph = Graph()
20 graph.add_vertex('A')
21 graph.add_vertex('B')
22 graph.add_edge('A', 'B')
23 print(graph)
```

```
/usr/local/bin/python3 "/Users/geethanjali/Desktop/AI/Graph Representation.py"
{A: ['B'], 'B': ['A']}
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code: class

PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```
monstrade.py | simple queue data.py | # linked list and .py | HashTable.py | Graph Representation.py | Priority Queue.py | Binary.py | #generate linked-1 |
Priority Queue.py > ...
1 #generate a code of data structure for Priority Queue
2
3 class PriorityQueue:
4     def __init__(self):
5         self.heap = []
6
7     def push(self, item, priority):
8         self.heap.append((priority, item))
9         self._heapify_up(len(self.heap) - 1)
10
11     def pop(self):
12         if not self.heap:
13             raise IndexError("pop from empty priority queue")
14         self._swap(0, len(self.heap) - 1)
15         item = self.heap.pop()
16         self._heapify_down(0)
17         return item[1]
18
19     def peek(self):
20         if not self.heap:
21             raise IndexError("peek from empty priority queue")
22         return self.heap[0][1]
23
24     def _heapify_up(self, index):
25         parent_index = (index - 1) // 2
26         if index > 0 and self.heap[index][0] < self.heap[parent_index][0]:
27             self._swap(index, parent_index)
28             self._heapify_up(parent_index)
29
30     def _heapify_down(self, index):
31         smallest = index
32         left_child = 2 * index + 1
33         right_child = 2 * index + 2
34
35         if left_child < len(self.heap) and self.heap[left_child][0] < self.heap[smallest][0]:
36             smallest = left_child
37         if right_child < len(self.heap) and self.heap[right_child][0] < self.heap[smallest][0]:
38             smallest = right_child
39         if smallest != index:
40             self._swap(index, smallest)
41             self._heapify_down(smallest)
42
43     def _swap(self, i, j):
44         self.heap[i], self.heap[j] = self.heap[j], self.heap[i]
45
46     def is_empty(self):
47         return len(self.heap) == 0
48
49     def size(self):
50         return len(self.heap)
51
52 # Example usage:
53 pq = PriorityQueue()
54 pq.push(1, 2)
55 pq.push(2, 1)
56 print(pq.pop()) # Output: "task2"
57 print(pq.peek()) # Output: "task1"
58 print(pq.is_empty()) # Output: False
59 print(pq.size()) # Output: 1
60 pq.pop()
61 print(pq.is_empty()) # Output: True
62 print(pq.size()) # Output: 0
63
```

```
Priority Queue.py > ...
30 def _heapify_down(self, index):
31     smallest = left_child
32     if right_child < len(self.heap) and self.heap[right_child][0] < self.heap[smallest][0]:
33         smallest = right_child
34     if smallest != index:
35         self._swap(index, smallest)
36         self._heapify_down(smallest)
37
38     def _swap(self, i, j):
39         self.heap[i], self.heap[j] = self.heap[j], self.heap[i]
40
41     def is_empty(self):
42         return len(self.heap) == 0
43
44     def size(self):
45         return len(self.heap)
46
47 # Example usage:
48 pq = PriorityQueue()
49 pq.push("task1", 2)
50 pq.push("task2", 1)
51 print(pq.pop()) # Output: "task2"
52 print(pq.peek()) # Output: "task1"
53 print(pq.is_empty()) # Output: False
54 print(pq.size()) # Output: 1
55 pq.pop()
56 print(pq.is_empty()) # Output: True
57 print(pq.size()) # Output: 0
58
59
```

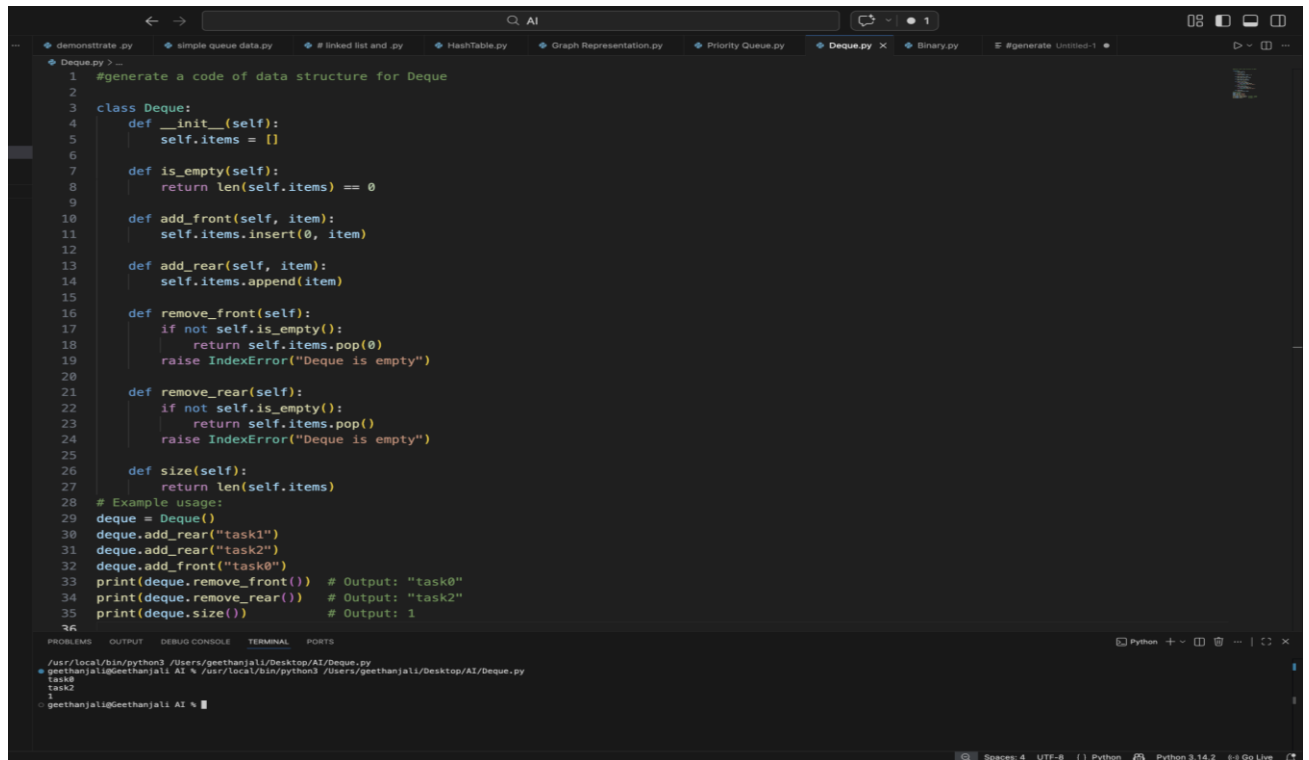
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python +
/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/Priority Queue.py
geethanjali@Geethanjali AI % /usr/local/bin/python3 /Users/geethanjali/Desktop/AI/Priority Queue.py
task2
task1
False
1
True
0
geethanjali@Geethanjali AI %
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`. Sample Input Code: `class DequeDS:`
`pass`

Expected Output:

- Insert and remove from both ends with docstrings.



```
1 #generate a code of data structure for Deque
2
3 class Deque:
4     def __init__(self):
5         self.items = []
6
7     def is_empty(self):
8         return len(self.items) == 0
9
10    def add_front(self, item):
11        self.items.insert(0, item)
12
13    def add_rear(self, item):
14        self.items.append(item)
15
16    def remove_front(self):
17        if not self.is_empty():
18            return self.items.pop(0)
19        raise IndexError("Deque is empty")
20
21    def remove_rear(self):
22        if not self.is_empty():
23            return self.items.pop()
24        raise IndexError("Deque is empty")
25
26    def size(self):
27        return len(self.items)
28
29 # Example usage:
30 deque = Deque()
31 deque.add_rear("task1")
32 deque.add_rear("task2")
33 deque.add_front("task0")
34 print(deque.remove_front()) # Output: "task0"
35 print(deque.remove_rear()) # Output: "task2"
36 print(deque.size()) # Output: 1
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o

Queue

o Priority

Queue o Linked

List o Binary

Search Tree (BST)

o Graph o Hash

Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

439 # generate a code for Cafeteria Order Queue - Serve students in the order they arrive using a queue data
440 class CafeteriaOrderQueue:
441     def __init__(self):
442         """Initialize the cafeteria order queue with an empty list."""
443         self.orders = []
444     def enqueue(self, student_name):
445         """Add a student's order to the end of the queue."""
446         self.orders.append(student_name)
447         print(f'Enqueued order for {student_name}')
448     def dequeue(self):
449         """Serve the next student in the order they arrived (remove from the front of the queue)."""
450         if not self.is_empty():
451             student_name = self.orders.pop(0)
452             print(f'Served {student_name}')
453             return student_name
454         else:
455             print('No orders to serve. The queue is empty.')
456             return None
457     def is_empty(self):
458         """Check if there are no orders in the queue."""
459         return len(self.orders) == 0
460     def display(self):
461         """Display the current orders in the queue."""
462         print('Current Orders in Queue:', self.orders)
463 # Demonstration of cafeteria order queue operations
464 if __name__ == "__main__":
465     order_queue = CafeteriaOrderQueue()
466     order_queue.enqueue('Alice')
467     order_queue.enqueue('Bob')
468     order_queue.enqueue('Charlie')
469     order_queue.display()
470     order_queue.dequeue()
471     order_queue.display()
472     order_queue.dequeue()
473     order_queue.dequeue()
474     order_queue.dequeue() # Attempt to serve from an empty queue
475     order_queue.display() # Display the empty queue
476

```

```

ive/ドキュメント/third/AIAC/lab_assignment_14.py
Enqueued order for Alice
Enqueued order for Bob
Enqueued order for Charlie
Current Orders in Queue: ['Alice', 'Bob', 'Charlie']
Served Alice
Current Orders in Queue: ['Bob', 'Charlie']
Served Bob
Served Charlie
No orders to serve. The queue is empty.
Current Orders in Queue: []
○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Feature	Chosen Data Structure	Justification
Student Attendance Tracking	Hash Table	A hash table allows fast insertion and lookup of student entries/exits by ID. It ensures efficient daily logging and quick verification of whether a student is currently on campus.

Feature	Chosen Data Structure	Justification
Event Registration System	Binary Search Tree (BST)	A BST supports ordered storage of participants, enabling quick search, insertion, and removal. It is ideal for managing event participants where alphabetical or ID-based ordering is useful.
Library Book Borrowing	Priority Queue	A priority queue can track books by due dates, ensuring that the most urgent returns (earliest due dates) are prioritized. This helps librarians manage overdue books efficiently.
Bus Scheduling System	Graph	Bus routes and stop connections naturally form a graph, where stops are nodes and routes are edges. Graph traversal algorithms can be used to find shortest paths or optimize scheduling.
Cafeteria Order Queue	Queue	A queue ensures students are served in the order they arrive (FIFO principle). This matches the real-world cafeteria process where fairness and order are maintained.

A Queue is the best fit because it follows the First-In-First-Out (FIFO) principle. This means students are served in the exact order they arrive, ensuring fairness and efficiency. Just like in a real cafeteria line, the first student to place an order will be the first to get served.

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o

Queue

o Priority

Queue o Linked

List o Binary

Search Tree (BST)

o Graph o Hash

Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
6 class HospitalManagementSystem:
7     def __init__(self):
8         """Initialize the hospital management system with an empty patient queue."""
9         self.patient_queue = []
10    def check_in(self, patient_name):
11        """Register a patient and add them to the end of the queue."""
12        self.patient_queue.append(patient_name)
13        print(f'Checked in patient: {patient_name}')
14    def treat_patient(self):
15        """Treat the next patient in the order they arrived (remove from the front of the queue)."""
16        if not self.is_empty():
17            patient_name = self.patient_queue.pop(0)
18            print(f'Treated patient: {patient_name}')
19            return patient_name
20        else:
21            print('No patients to treat. The queue is empty.')
22            return None
23    def is_empty(self):
24        """Check if there are no patients in the queue."""
25        return len(self.patient_queue) == 0
26    def display_queue(self):
27        """Display the current patients in the queue."""
28        print('Current Patients in Queue:', self.patient_queue)
29    # Demonstration of hospital management system operations
30    if __name__ == "__main__":
31        hospital_system = HospitalManagementSystem()
32        hospital_system.check_in('John Doe')
33        hospital_system.check_in('Jane Smith')
34        hospital_system.check_in('Emily Davis')
35        hospital_system.display_queue()
36        hospital_system.treat_patient()
37        hospital_system.display_queue()
38        hospital_system.treat_patient()
39        hospital_system.treat_patient()
40        hospital_system.treat_patient() # Attempt to treat from an empty queue
41        hospital_system.display_queue() # Display the empty queue
```

```
Checked in patient: John Doe
Checked in patient: Jane Smith
Checked in patient: Emily Davis
Current Patients in Queue: ['John Doe', 'Jane Smith', 'Emily Davis']
Treated patient: John Doe
Current Patients in Queue: ['Jane Smith', 'Emily Davis']
Treated patient: Jane Smith
Treated patient: Emily Davis
No patients to treat. The queue is empty.
Current Patients in Queue: []
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Feature	Data Structure	Justification
Patient Check-In System	Queue	Patients are treated in the order they arrive, which follows the FIFO (First-In-First-Out) principle. A queue ensures fair and sequential handling.
Emergency Case Handling	Priority Queue	Critical patients must be treated first. A priority queue allows assigning urgency levels so higher-priority patients are handled before others.
Medical Records Storage	Hash Table	Patient details can be quickly retrieved using their unique ID. Hash tables provide $O(1)$ average-time lookup.
Doctor Appointment Scheduling	Linked List	Appointments can be stored in sorted order by time. A linked list allows easy insertion and traversal in chronological order.
Hospital Room Navigation	Graph	Wards and rooms are interconnected. A graph represents these connections, enabling efficient navigation and shortest path finding.

Patients must be treated in the exact order they arrive. A queue ensures fairness by following the FIFO principle, so no one skips ahead unless marked as an emergency case (which would use a priority queue instead).

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
524 class TrafficSignalQueue:
525     def __init__(self):
526         """Initialize the traffic signal queue with an empty list."""
527         self.vehicle_queue = []
528     def arrive(self, vehicle_number):
529         """Add a vehicle to the end of the queue when it arrives at the traffic signal."""
530         self.vehicle_queue.append(vehicle_number)
531         print(f'Vehicle {vehicle_number} arrived at the traffic signal')
532     def pass_signal(self):
533         """Allow the next vehicle in the order they arrived to pass through the traffic signal (remove from the front of the queue)."""
534         if not self.is_empty():
535             vehicle_number = self.vehicle_queue.pop(0)
536             print(f'Vehicle {vehicle_number} passed through the traffic signal')
537             return vehicle_number
538         else:
539             print('No vehicles to pass. The queue is empty.')
540             return None
541     def is_empty(self):
542         """Check if there are no vehicles in the queue."""
543         return len(self.vehicle_queue) == 0
544     def display_queue(self):
545         """Display the current vehicles waiting at the traffic signal."""
546         print('Current Vehicles in Queue:', self.vehicle_queue)
547 # Demonstration of traffic signal queue operations
548 if __name__ == "__main__":
549     traffic_queue = TrafficSignalQueue()
550     traffic_queue.arrive('ABC123')
551     traffic_queue.arrive('XYZ789')
552     traffic_queue.arrive('LMN456')
553     traffic_queue.display_queue()
554     traffic_queue.pass_signal()
555     traffic_queue.display_queue()
556     traffic_queue.pass_signal()
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
Vehicle ABC123 arrived at the traffic signal
Vehicle XYZ789 arrived at the traffic signal
Vehicle LMN456 arrived at the traffic signal
Current Vehicles in Queue: ['ABC123', 'XYZ789', 'LMN456']
Vehicle ABC123 passed through the traffic signal
Current Vehicles in Queue: ['XYZ789', 'LMN456']
Vehicle XYZ789 passed through the traffic signal
Vehicle LMN456 passed through the traffic signal
No vehicles to pass. The queue is empty.
Current Vehicles in Queue: []
```

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	Vehicles wait at signals in the order they arrive. A queue follows the FIFO principle, ensuring fair passage.
Emergency Vehicle Priority Handling	Priority Queue	Ambulances and fire trucks must be prioritized. A priority queue ensures vehicles with higher urgency are processed first.
Vehicle Registration Lookup	Hash Table	Vehicle details can be quickly retrieved using registration numbers. Hash tables provide O(1) average lookup time.
Road Network Mapping	Graph	Roads and intersections form a network. A graph represents connections, enabling shortest path and route planning.
Feature	Data Structure	Justification

Parking Slot Availability	Binary Search Tree (BST)	Parking slots can be tracked as available/occupied. A BST allows efficient searching, insertion, and deletion of slots by number.
---------------------------	--------------------------	---

Vehicles at a traffic signal must move in the order they arrive. A queue ensures fairness and follows the FIFO principle, so no vehicle skips ahead unless marked as an emergency (which would use a priority queue).

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o

Queue

o Priority

Queue o Linked

List o Binary

Search Tree (BST)

o Graph o Hash

Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.


```

562 class ShoppingCart:
563     def __init__(self):
564         """Initialize the shopping cart with an empty list of products."""
565         self.cart = []
566     def add_product(self, product_name):
567         """Add a product to the shopping cart."""
568         self.cart.append(product_name)
569         print(f'Added {product_name} to the shopping cart')
570     def remove_product(self, product_name):
571         """Remove a product from the shopping cart if it exists."""
572         if product_name in self.cart:
573             self.cart.remove(product_name)
574             print(f'Removed {product_name} from the shopping cart')
575         else:
576             print(f'Product {product_name} not found in the shopping cart. Cannot remove.')
577     def display_cart(self):
578         """Display the current products in the shopping cart."""
579         print('Current Shopping Cart:', self.cart)
580 # Demonstration of shopping cart management operations
581 if __name__ == "__main__":
582     shopping_cart = ShoppingCart()
583     shopping_cart.add_product('Laptop')
584     shopping_cart.add_product('Smartphone')
585     shopping_cart.add_product('Headphones')
586     shopping_cart.display_cart()
587     shopping_cart.remove_product('Smartphone')
588     shopping_cart.display_cart()
589     shopping_cart.remove_product('Tablet') # Attempt to remove a non-existent product
590     shopping_cart.display_cart() # Display the current state of the cart
591

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

ive/ドキュメント/third/AIAC/lab_assignment_14.py
Added Laptop to the shopping cart
Added Smartphone to the shopping cart
Added Headphones to the shopping cart
Current Shopping Cart: ['Laptop', 'Smartphone', 'Headphones']
Removed Smartphone from the shopping cart
Current Shopping Cart: ['Laptop', 'Headphones']
Product Tablet not found in the shopping cart. Cannot remove.
Current Shopping Cart: ['Laptop', 'Headphones']

```

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Products can be added or removed dynamically. A linked list supports efficient insertion and deletion without shifting elements.
Order Processing System	Queue	Orders must be processed in the order they are placed. A queue ensures FIFO (First-In-First-Out) handling.
Top-Selling Products Tracker	Priority Queue	Products can be ranked by sales count. A priority queue ensures the highest-selling products are always accessible first.

Product Search Engine	Hash Table	Products can be quickly retrieved using product IDs. Hash tables provide $O(1)$ average lookup time.
Feature	Data Structure	Justification
Delivery Route Planning	Graph	Warehouses and delivery locations form a network. A graph represents connections and supports shortest path algorithms.

A shopping cart needs dynamic addition and removal of products. A linked list is ideal because it allows efficient insertion and deletion without shifting elements, unlike arrays.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:

☐ Stack ☐

Queue

☐ Priority

Queue ☐ Linked

List ☐ Binary

Search Tree (BST)

☐ Graph ☐ Hash

Table ☐ Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

594 class ShipmentProcessing:
595     def __init__(self):
596         """Initialize the shipment processing system with an empty list of shipments."""
597         self.shipments = []
598     def add_shipment(self, shipment_id):
599         """Add a shipment to the processing queue."""
600         self.shipments.append(shipment_id)
601         print(f'Added shipment {shipment_id} to the processing queue')
602     def process_shipment(self):
603         """Process the next shipment in the order they were added (remove from the front of the queue)."""
604         if not self.is_empty():
605             shipment_id = self.shipments.pop(0)
606             print(f'Processed shipment {shipment_id}')
607             return shipment_id
608         else:
609             print('No shipments to process. The queue is empty.')
610             return None
611     def is_empty(self):
612         """Check if there are no shipments in the processing queue."""
613         return len(self.shipments) == 0
614     def display_queue(self):
615         """Display the current shipments in the processing queue."""
616         print('Current Shipments in Queue:', self.shipments)
617 # Demonstration of shipment processing operations
618 if __name__ == "__main__":
619     shipment_processing = ShipmentProcessing()
620     shipment_processing.add_shipment('SHIP001')
621     shipment_processing.add_shipment('SHIP002')
622     shipment_processing.add_shipment('SHIP003')
623     shipment_processing.display_queue()
624     shipment_processing.process_shipment()
625     shipment_processing.display_queue()
626     shipment_processing.process_shipment()

```

Feature	Data Structure	Justification
Shipment Processing	Queue	Shipments are processed in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling.
Urgent Shipment Priority	Priority Queue	Urgent shipments must be handled first. A priority queue allows assigning urgency levels so critical shipments are processed before normal ones.
Product Lookup by Code	Hash Table	Products can be quickly retrieved using their unique code. Hash tables provide $O(1)$ average lookup time.

Warehouse Network Connectivity	Graph	Warehouses and routes form a connected network. A graph represents these connections and supports shortest path algorithms for efficient delivery.
Feature	Data Structure	Justification
Inventory Add/Remove Operations	Linked List	Inventory needs dynamic addition and removal of products. A linked list supports efficient insertion and deletion without shifting elements.

Shipments must be processed in the order they arrive. A queue ensures fairness and follows the FIFO principle, so no shipment is skipped unless marked urgent (which would be handled by a priority queue).

Task Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

2 class CustomerServiceTokenSystem:
5     self.token_queue = []
6     def take_token(self, customer_name):
7         """Assign a token to a customer and add them to the end of the queue."""
8         self.token_queue.append(customer_name)
9         print(f'{customer_name} took a token and is added to the queue')
10    def serve_customer(self):
11        """Serve the next customer in the order they arrived (remove from the front of the queue)."""
12        if not self.is_empty():
13            customer_name = self.token_queue.pop(0)
14            print(f'Served customer: {customer_name}')
15            return customer_name
16        else:
17            print('No customers to serve. The queue is empty.')
18            return None
19    def is_empty(self):
20        """Check if there are no customers in the queue."""
21        return len(self.token_queue) == 0
22    def display_queue(self):
23        """Display the current customers waiting in the queue."""
24        print('Current Customers in Queue:', self.token_queue)
25
26    # Demonstration of customer service token system operations
27    if __name__ == "__main__":
28        token_system = CustomerServiceTokenSystem()
29        token_system.take_token('Alice')
30        token_system.take_token('Bob')
31        token_system.take_token('Charlie')
32        token_system.display_queue()
33        token_system.serve_customer()
34        token_system.display_queue()
35        token_system.serve_customer()
36        token_system.serve_customer()

```

PROBLEMS

OUTPUT

TERMINAL

PORTS

```

7 TERMINAL
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python311/Python.exe /d
ive/ドキュメント/third/AIAC/lab_assignment_14.py
Bob took a token and is added to the queue
Charlie took a token and is added to the queue
Current Customers in Queue: ['Alice', 'Bob', 'Charlie']
Served customer: Alice
Current Customers in Queue: ['Bob', 'Charlie']
Served customer: Bob
Served customer: Charlie
No customers to serve. The queue is empty.
Current Customers in Queue: []

```

Feature	Data Structure	Justification
Customer Service Token System	Queue	Customers are served in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling, maintaining fairness.
Loan Approval Priority Handling	Priority Queue	High-value or urgent loans must be processed first. A priority queue allows assigning priority levels so urgent loans are handled before regular ones.
Account Information Lookup	Hash Table	Customer details can be quickly retrieved using account numbers. Hash tables provide O(1) average lookup time.

Feature	Data Structure	Justification
Transaction History Management	Linked List	Transactions are stored chronologically. A linked list allows efficient insertion at the end and traversal in order.
ATM Network Connectivity	Graph	ATMs across cities form a connected network. A graph represents these connections and supports shortest path algorithms for efficient routing.

Customers must be served in the order they arrive. A queue ensures fairness by following the FIFO principle, so no one skips ahead unless marked as urgent (which would use a priority queue).

Task Description #15: Online Learning Management System – Data Structure Selection An

online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.


```

694 # generate a code to demonstrate an airport authority plans to implement: Passenger Check-In Queue - Process
695 class PassengerCheckInQueue:
696     def __init__(self):
697         """Initialize the passenger check-in queue with an empty list."""
698         self.passenger_queue = []
699     def check_in(self, passenger_name):
700         """Add a passenger to the end of the check-in queue."""
701         self.passenger_queue.append(passenger_name)
702         print(f'Checked in passenger: {passenger_name}')
703     def process_passenger(self):
704         """Process the next passenger in the order they arrived (remove from the front of the queue)."""
705         if not self.is_empty():
706             passenger_name = self.passenger_queue.pop(0)
707             print(f'Processed passenger: {passenger_name}')
708             return passenger_name
709         else:
710             print('No passengers to process. The queue is empty.')
711             return None
712     def is_empty(self):
713         """Check if there are no passengers in the queue."""
714         return len(self.passenger_queue) == 0
715     def display_queue(self):
716         """Display the current passengers in the check-in queue."""
717         print('Current Passengers in Queue:', self.passenger_queue)
718 # Demonstration of passenger check-in queue operations
719 if __name__ == "__main__":
720     check_in_queue = PassengerCheckInQueue()
721     check_in_queue.check_in('John Doe')
722     check_in_queue.check_in('Jane Smith')
723     check_in_queue.check_in('Emily Davis')
724     check_in_queue.display_queue()
725     check_in_queue.process_passenger()

```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/py
Checked in passenger: Emily Davis
Current Passengers in Queue: ['John Doe', 'Jane Smith', 'Emily Davis']
Processed passenger: John Doe
Current Passengers in Queue: ['Jane Smith', 'Emily Davis']
Processed passenger: Jane Smith
Processed passenger: Emily Davis
No passengers to process. The queue is empty.
Current Passengers in Queue: []

```

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	Passengers must be processed in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling, maintaining fairness.
Emergency Landing Priority	Priority Queue	Aircraft in emergency situations must be prioritized. A priority queue ensures urgent landings are handled before regular ones.
Flight Information Lookup	Hash Table	Flight details can be quickly retrieved using flight IDs. Hash tables provide O(1) average lookup time.
Air Route Connectivity	Graph	Airports and routes form a connected network. A graph represents these connections and supports shortest path algorithms for efficient navigation.
Boarding Process (Cabin Storage)	Stack	Overhead luggage follows LIFO (Last-In-First-Out) since the last bag placed is the first one removed. A stack is ideal for simulating this process.

Overhead luggage storage works in a LIFO manner—the last bag placed is the first one removed. A stack perfectly models this behavior, making it the most suitable data structure.

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

```

733 class NewsFeed:
734     def __init__(self):
735         """Initialize the news feed with an empty list of posts."""
736         self.posts = []
737     def add_post(self, post_content):
738         """Add a new post to the news feed."""
739         self.posts.append(post_content)
740         print(f'Added post: {post_content}')
741     def display_feed(self):
742         """Display the news feed with posts in chronological order."""
743         print('News Feed:')
744         for post in self.posts:
745             print(post)
746 # Demonstration of news feed generation operations
747 if __name__ == "__main__":
748     news_feed = NewsFeed()
749     news_feed.add_post('Post 1: Hello World!')
750     news_feed.add_post('Post 2: Welcome to the social media platform.')
751     news_feed.add_post('Post 3: Enjoy your stay!')
752     news_feed.display_feed()
753

```

PROBLEMS

OUTPUT

TERMINAL

PORTS

▼ TERMINAL

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local
ive/ドキュメント/third/AIAC/lab_assignment_14.py
Added post: Post 1: Hello World!
Added post: Post 2: Welcome to the social media platform.
Added post: Post 3: Enjoy your stay!
News Feed:
Post 1: Hello World!
Post 2: Welcome to the social media platform.
Post 3: Enjoy your stay!

Feature	Data Structure	Justification
News Feed Generation	Linked List	Posts are displayed chronologically. A linked list allows efficient insertion at the end and traversal in order.
Trending Hashtags Tracker	Priority Queue	Hashtags must be ranked by usage frequency. A priority queue ensures the most frequently used hashtags are always accessible first.
User Profile Search	Hash Table	User profiles can be quickly retrieved using usernames. Hash tables provide O(1) average lookup time.
Friend Network Representation	Graph	Friend connections form a network. A graph represents relationships and supports traversal for mutual connections or recommendations.

Feature	Data Structure	Justification
Message Undo Feature	Stack	Undo requires reversing the most recent action. A stack follows LIFO (Last-In-First-Out) , making it ideal for withdrawing the last sent message.

Undo requires reversing the most recent action. A stack is perfect because it follows the LIFO principle, ensuring the last message sent is the first one withdrawn.

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry

```

lab_assignment_14.py 2 ...
755 class HostelManagementSystem:
756     def __init__(self):
757         """Initialize the hostel management system with an empty list of rooms."""
758         self.rooms = []
759     def allot_room(self, student_name):
760         """Allot a room to a student and add them to the list of rooms."""
761         self.rooms.append(student_name)
762         print(f'Allotted room to {student_name}')
763     def vacate_room(self, student_name):
764         """Vacate a room from a student if they exist."""
765         if student_name in self.rooms:
766             self.rooms.remove(student_name)
767             print(f'Vacated room from {student_name}')
768         else:
769             print(f'Student {student_name} not found in the hostel. Cannot vacate room.')
770     def display_rooms(self):
771         """Display the current students allotted rooms in the hostel."""
772         print('Current Rooms Allotted:', self.rooms)
773 # Demonstration of hostel management system operations
774 if __name__ == "__main__":
775     hostel_system = HostelManagementSystem()
776     hostel_system.allot_room('Alice')
777     hostel_system.allot_room('Bob')
778     hostel_system.allot_room('Charlie')
779     hostel_system.display_rooms()
780     hostel_system.vacate_room('Bob')
781     hostel_system.display_rooms()
782     hostel_system.vacate_room('David') # Attempt to vacate a non-existent student
783     hostel_system.display_rooms() # Display the current state of rooms allotted

```

PROBLEMS OUTPUT TERMINAL PORTS

> **TERMINAL**

```

Allotted room to Alice
Allotted room to Bob
Allotted room to Charlie
Current Rooms Allotted: ['Alice', 'Bob', 'Charlie']
Vacated room from Bob
Current Rooms Allotted: ['Alice', 'Charlie']
Student David not found in the hostel. Cannot vacate room.
Current Rooms Allotted: ['Alice', 'Charlie']

```

Feature	Data Structure	Justification
Room Allotment	Binary Search Tree (BST)	Rooms can be allotted based on room numbers. A BST allows efficient searching, insertion, and deletion of rooms in sorted order.
Medical Emergency Handling	Priority Queue	Emergency cases must be handled first. A priority queue ensures urgent medical needs are prioritized over normal requests.
Student Record Retrieval	Hash Table	Student details can be quickly retrieved using unique IDs. Hash tables provide O(1) average lookup time.
Hostel Block Connectivity	Graph	Hostel blocks and rooms form a connected network. A graph represents these connections and supports navigation or shortest path finding.
Visitor Entry Log	Queue	Visitors are logged in the order they arrive. A queue ensures FIFO (First-In-First-Out) handling of entries.

Visitors must be logged and processed in the order they arrive. A queue ensures fairness by following the FIFO principle, so no visitor is skipped ahead.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking • Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.


```

786 class ComplaintHandlingSystem:
787     def __init__(self):
788         """Initialize the complaint handling system with an empty list of complaints."""
789         self.complaints = []
790     def file_complaint(self, customer_name, complaint_details):
791         """File a new complaint and add it to the list of complaints."""
792         complaint = {'customer_name': customer_name, 'complaint_details': complaint_details}
793         self.complaints.append(complaint)
794         print(f'Filed complaint from {customer_name}: {complaint_details}')
795     def resolve_complaint(self):
796         """Resolve the next complaint in the order they were filed (remove from the front of the list)."""
797         if not self.is_empty():
798             complaint = self.complaints.pop(0)
799             print(f'Resolved complaint from {complaint["customer_name"]}: {complaint["complaint_details"]}')
800             return complaint
801         else:
802             print('No complaints to resolve. The list is empty.')
803             return None
804     def is_empty(self):
805         """Check if there are no complaints in the system."""
806         return len(self.complaints) == 0
807     def display_complaints(self):
808         """Display the current complaints in the system."""
809         print('Current Complaints:')
810         for complaint in self.complaints:
811             print(f'{complaint["customer_name"]}: {complaint["complaint_details"]}')
812 if __name__ == "__main__":
813     complaint_system = ComplaintHandlingSystem()
814     complaint_system.file_complaint('Alice', 'Power outage in my area')
815     complaint_system.file_complaint('Bob', 'Billing issue with my last bill')
816     complaint_system.file_complaint('Charlie', 'Faulty meter reading')
817     complaint_system.display_complaints()
818     complaint_system.resolve_complaint()
819     complaint_system.display_complaints()
820     complaint_system.resolve_complaint()
821     complaint_system.resolve_complaint()
822     complaint_system.resolve_complaint() # Attempt to resolve from an empty list
823     complaint_system.display_complaints() # Display the empty list of complaints

```

```

Filed complaint from Alice: Power outage in my area
Filed complaint from Bob: Billing issue with my last bill
Filed complaint from Charlie: Faulty meter reading
Current Complaints:
Alice: Power outage in my area
Bob: Billing issue with my last bill
Charlie: Faulty meter reading
Resolved complaint from Alice: Power outage in my area
Current Complaints:
Bob: Billing issue with my last bill
Charlie: Faulty meter reading
Resolved complaint from Bob: Billing issue with my last bill
Resolved complaint from Charlie: Faulty meter reading
No complaints to resolve. The list is empty.
Current Complaints:

```

Feature	Data Structure	Justification
Complaint Handling	Queue	Complaints must be addressed in the order they are received. A queue ensures FIFO (First-In-First-Out) processing, maintaining fairness.
Fault Priority Handling	Priority Queue	Critical faults must be resolved first. A priority queue allows assigning urgency levels so severe faults are handled before minor ones.

Feature	Data Structure	Justification
Consumer Record Lookup	Hash Table	Consumer details can be quickly retrieved using unique IDs. Hash tables provide $O(1)$ average lookup time.
Power Grid Connectivity	Graph	Power grids and substations form a connected network. A graph represents these connections and supports shortest path or flow analysis.
Meter Reading History Tracking	Linked List	Meter readings are stored chronologically. A linked list allows efficient insertion at the end and traversal in order.

Complaints must be processed in the order they are received. A queue ensures fairness by following the FIFO principle, so no complaint is skipped ahead unless marked urgent (which would be handled by a priority queue).